An Implementation of Treap &

Randomised Binary Search Trees:

A Complexity and Performance Analysis

Adam **Kearns** (23326903), Cormac **Kehoe** (23411302), William **Lin** (23314213), Liam **Maher** (23444882)

University College Dublin, Dublin, Ireland

Introduction

TREAPS

A *Treap* is a binary tree whose nodes contain both an *Entry<Key, Value>* and a *priority*. It combines properties of both a *Binary Search Tree* (abbrv. BST) and a *Heap*.

- *Treaps* maintains their *BST* property with respect to their keys. The *BST* property means that for every *node(x)*, *node(x).key* is greater than all keys in its left subtree, and less than all keys in its right subtree.
- *Treaps* maintain their *Heap* property with respect to their priorities. The heap property means that for any *node(x)*, its priority is less than or equal to its parent. When a *TreapNode* is created, it is assigned a priority **at random**.

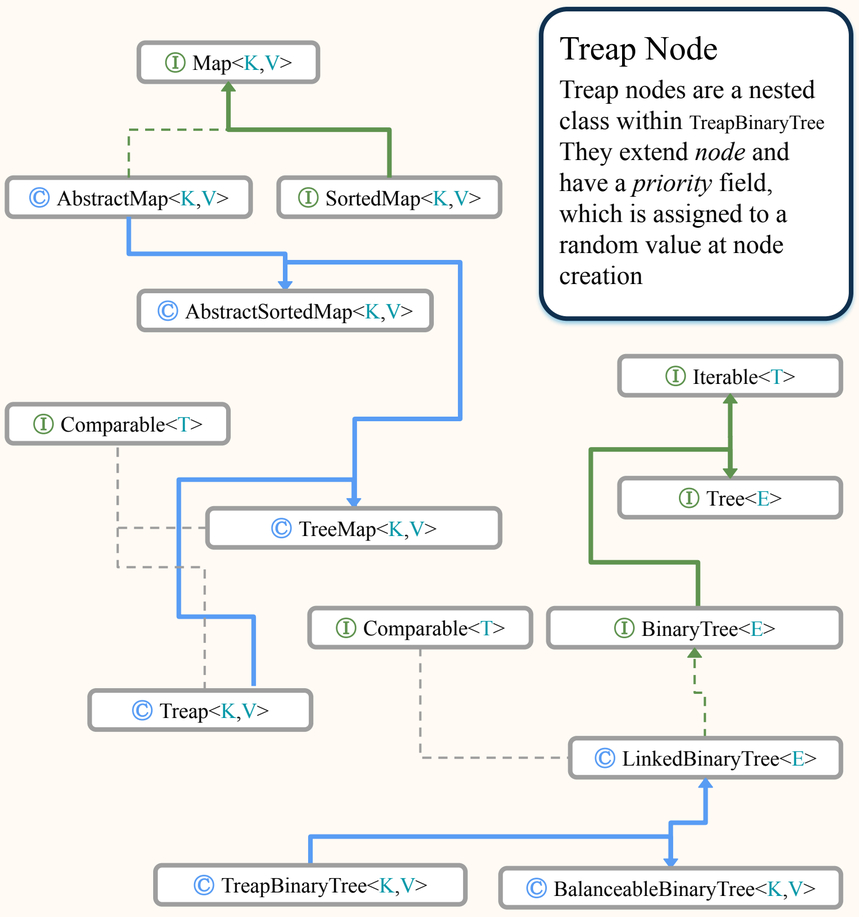
Class Diagram

HOW DOES TREAP WORK?

Treap simply extends an existing *TreeMap* class, which is similar to java.utils TreeMap, with some minor adjustments for our own implementation.

The treap class overrides the *remove( )* and *rebalanceInsert( )* of *TreeMap* to allow the functionality of the treap with regards to the heap functionality. When a node is added, it is added as a leaf similar to a normal BST. However, it is then upheaped based on its priority to maintain the heap property. This upheaping does not affect the BST property as the upheaping is done by utilising tree rotations.

For *remove( )*, we simply do the opposite. If the node is already a leaf we just remove it, and, if not, we then must downheap the node using tree rotations until it becomes a leaf node, thus keeping the BST and heap properties respectively. Once a node has been made a leaf, we unlink it from its parent, in turn removing it from the tree.



Treap Node

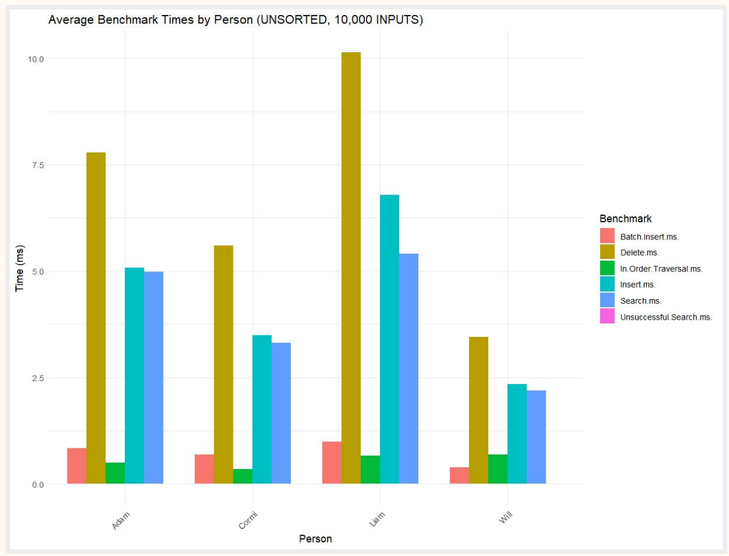
Treap nodes are a nested class within TreapBinaryTree They extend *node* and have a *priority* field, which is assigned to a random value at node creation

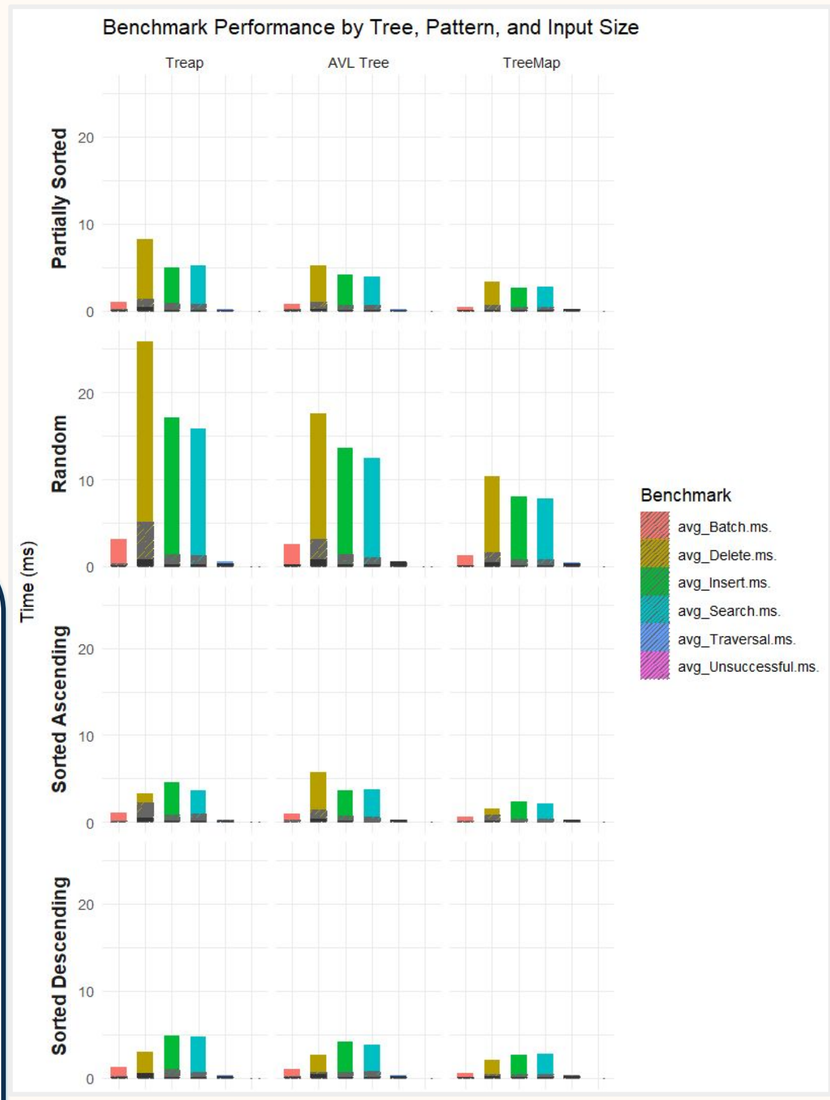
Data Structure	*Input Size*	*TreapSort( )*	*PQSort( )*	**JavaSort( )* **JavaSort() is Collections.sort()*	*QuickSort( )*	*MergeSort( )*
Random	100	2.73828	0.11108	0.11918	0.08682	0.07266
	1,000	1.19993	0.19274	0.23675	0.15092	0.22454
	10,000	5.64258	1.40117	0.94335	1.26442	1.22539
Nearly Sorted	100	0.50858	0.09461	0.04669	0.07359	0.04878
	1,000	0.84228	0.20510	0.08828	0.11908	0.25794
	10,000	3.99161	1.03581	0.59179	0.88487	0.91029
Reverse Sort	100	0.40290	0.05893	0.02374	0.05654	0.03873
	1,000	0.42897	0.21198	0.03622	0.14402	0.20021
	10,000	2.19429	0.98958	0.06777	0.89144	0.75424

~ Sorting Results Table ~

Results above are in milliseconds (ms) - Results are an average of 30 trials

Comparative Results Analysis





KEY FINDINGS

Based on our results, we can confidently say the following about *Treaps*:

- Each insertion into a Treap has a time complexity of O(*h*), with h being the height of the tree. Due to the randomisation of the insertions (being based on the nodes priorities), we reason that the expected height of the tree will be O(*log n*). If creating a treap from scratch with n inputs, this means it will be expected to have a complexity of O(*n log n*).
- For Treap deletions, we can also prove that it will have a time complexity of O(*h*) to remove a node from said Treap, which again (by the same reasoning as above) will be of an expected O(*log n*) time complexity.

Both of these results have come from fact that trees rotations are constant time operations, and thus our only constraint is the height of the tree. This is the great advantage of randomised BSTs, due to self balancing, the height of a given tree is always expected to be O(*log n*).

Sorting Results Discussion

Across all tested input patterns and sizes, *TreapSort( )* consistently performed slower than *PQSort( )*, Java's built-in sort, *QuickSort( )*, and *MergeSort( )*. While *TreapSort( )* maintains both *BST* and *Heap* properties, the overhead from the node-based structure and randomized balancing results in longer execution times, especially on larger inputs. In contrast, array-based algorithms, *MergeSort( )* and *QuickSort( ),* leverage better memory locality and structure, yielding faster results overall.

References and Resources

1. "Randomised Search Trees - R. Seidel, C.R. Aragon" [1996]
2. Graphing Software - R studio
3. IDE & Language - IntelliJ IDEA w/Java

Liam Cormac Adam William

Published April 2025